UNIVERSIDAD DE GRANADA

DECSAI

Técnicas de los Sistemas Inteligentes

Grado en Informática

Curso 2025-26. Práctica 1
Algoritmos de Búsqueda en Profundidad y A*

Pablo Mesejo Santiago y David Criado Ramón

Departamento de Ciencias de la Computación e Inteligencia Artificial

http://decsai.ugr.es

Nodos vs Estados

Nodo	Estado
Estructura de datos utilizada por algoritmos de búsqueda para **representar un estado (+información de utilidad para la búsqueda)**.	**Descripción abstracta** de una **configuración del mundo**.
Ejemplo: estructura de datos que representa el estado y contiene un puntero al nodo padre, la acción que llevó del padre al hijo, el coste de haber llegado a este estado, etc.	Ejemplo: en ajedrez, un estado podría ser una configuración determinada del tablero en un momento concreto.

Generalmente, un nodo está asociado con un único estado, pero un mismo estado puede estar representado por diferentes nodos (a los que se ha llegado a través de una secuencia diferente de acciones (de diferente coste, p.ej.))

Pregunta clave en IA simbólica: ¿cómo representamos el mundo? Es decir, ¿cómo modelamos los posibles estados?

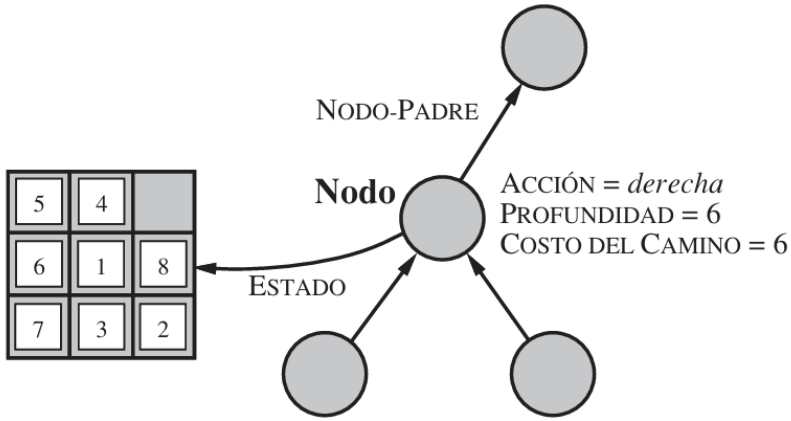
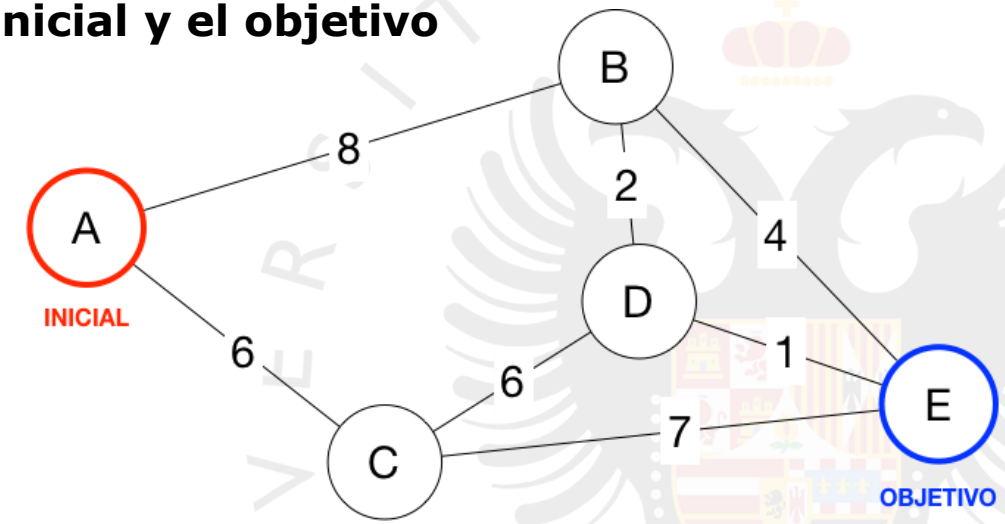


Figura 3.8 Los nodos son estructuras de datos a partir de los cuales se construye el árbol de búsqueda. Cada uno tiene un padre, un estado y varios campos. Las flechas señalan del hijo al padre.

Russell, S. J., & Norvig, P. (2004). *Artificial intelligence: a modern approach*. Pearson. Pág. 80. 2ª Edición

- **Input:**
  - **Un grafo**
  - **Un nodo inicial**
  - **Un nodo objetivo**
  - **Una función heurística (solo en búsqueda heurística)**
- **Output:**
  - **Una ruta entre el nodo inicial y el objetivo**



	A	B	C	D	E
h(n)	10	4	5	1	0

Búsqueda no informada

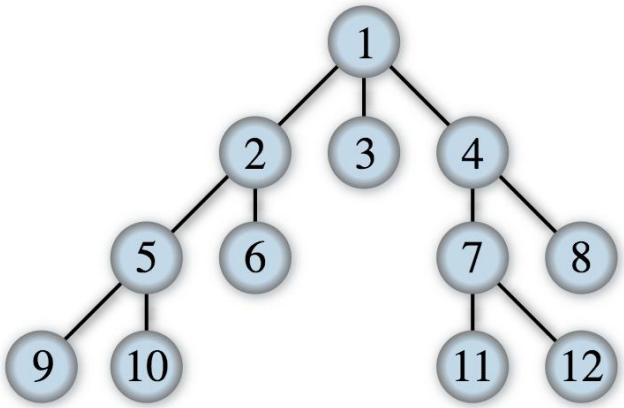
- La **búsqueda no informada** es aquella que **no usa** información heurística.

- Algunos ejemplos:
  - Búsqueda en anchura (BFS, *Breadth-first Search*).
  - Búsqueda en profundidad (DFS, *Depth-first Search*).
  - Algoritmo de Dijkstra
  - Descenso Iterativo (IDS, *Iterative Deepening Search*).

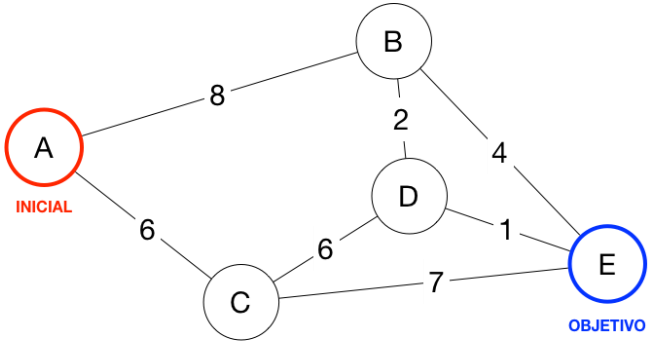
BFS & DFS:	[Russell & Norvig. Artificial Intelligence: A modern approach. Pearson, 2020]
Dijkstra:	[Dijkstra. 1959. Numerische Mathematik 1: 269-271]
IDS:	[Korf. 1985. Artificial Intelligence. 27: 97–109]

Búsqueda en anchura (BFS)

- Encuentra el camino óptimo (menor profundidad) cuando el coste de desplazamiento entre nodos es constante.
  - Si el coste es variable, no se garantiza el óptimo.
- Es muy ineficiente en tiempo y memoria.
  - Tiempo: expande muchas ramas que no son útiles.
  - Memoria: mantiene todo el árbol expandido.



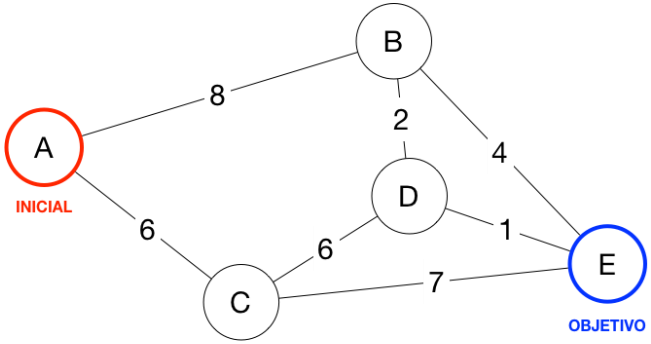
[S.J. Russell, P. Norvig. Artificial Intelligence: A modern approach. Pearson, 2020]



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido (nodo padre)	Cola	Estado				
			A	B	C	D	E
0	-	[A]	V	NV	NV	NV	NV

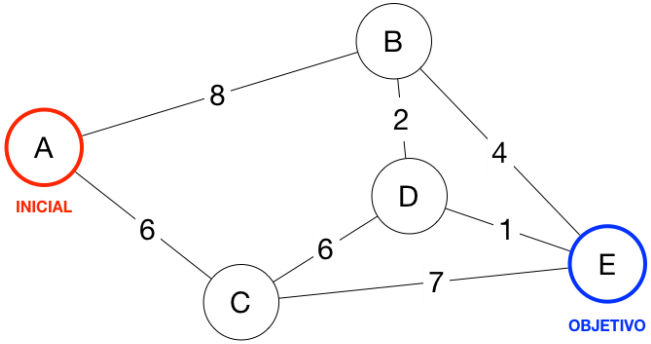
V: visitado / NV: no visitado
Orden de expansión: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido (nodo padre)	Cola	Estado				
			A	B	C	D	E
0	-	[A]	V	NV	NV	NV	NV
1	A (-)	[B,C]	V	V	V	NV	NV

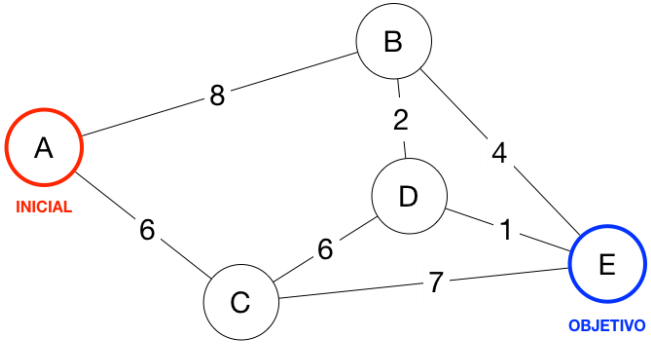
V: visitado / NV: no visitado
Orden de expansión: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido (nodo padre)	Cola	Estado				
			A	B	C	D	E
0	-	[A]	V	NV	NV	NV	NV
1	A (-)	[B,C]	V	V	V	NV	NV
2	B (A)	[C,D,E]	V	V	V	V	V

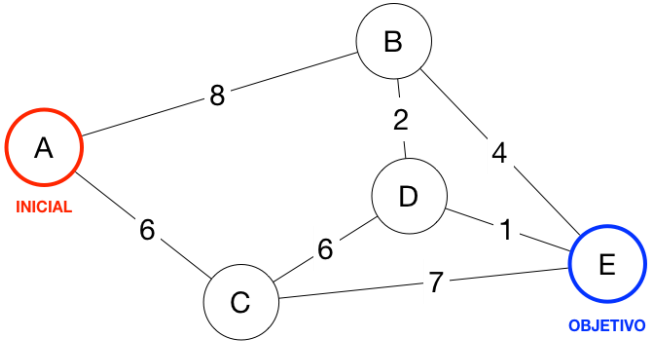
V: visitado / NV: no visitado
Orden de expansión: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido (nodo padre)	Cola	Estado				
			A	B	C	D	E
0	-	[A]	V	NV	NV	NV	NV
1	A (-)	[B,C]	V	V	V	NV	NV
2	B (A)	[C,D,E]	V	V	V	V	V
3	C (A)	[D,E]	V	V	V	V	V

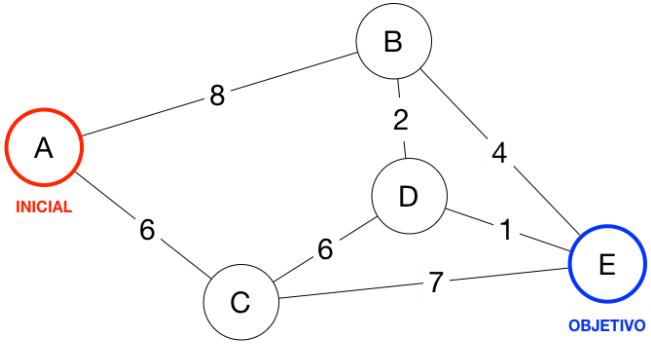
V: visitado / NV: no visitado
Orden de expansión: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido (nodo padre)	Cola	Estado				
			A	B	C	D	E
0	-	[A]	V	NV	NV	NV	NV
1	A (-)	[B,C]	V	V	V	NV	NV
2	B (A)	[C,D,E]	V	V	V	V	V
3	C (A)	[D,E]	V	V	V	V	V
4	D (B)	[E]	V	V	V	V	V

V: visitado / NV: no visitado
Orden de expansión: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

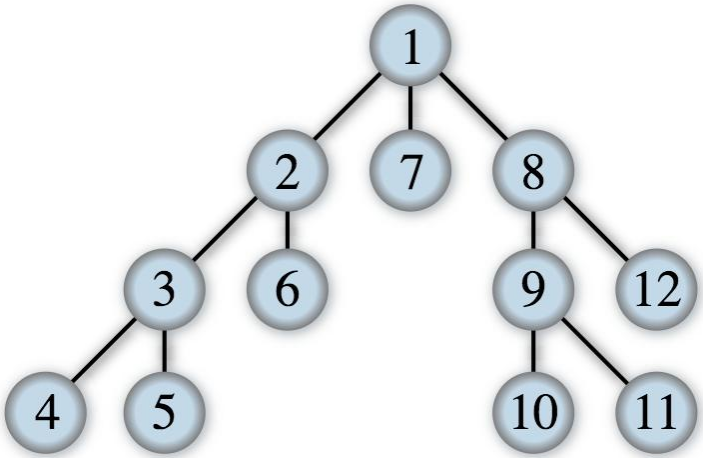
It.	Nodo expandido (nodo padre)	Cola	Estado				
			A	B	C	D	E
0	-	[A]	V	NV	NV	NV	NV
1	A (-)	[B,C]	V	V	V	NV	NV
2	B (A)	[C,D,E]	V	V	V	V	V
3	C (A)	[D,E]	V	V	V	V	V
4	D (B)	[E]	V	V	V	V	V
5	E (B)	Fin. Camino encontrado: E – B – A					

V: visitado / NV: no visitado
Orden de expansión: orden alfabético (A,B,C, …)

No es óptimo!!!
Porque el coste de los caminos no es uniforme

Búsqueda en profundidad (DFS)

- No garantiza encontrar el camino óptimo.
  - Incluso si el coste de desplazamiento entre nodos es constante.
- En el caso medio es más eficiente que BFS en tiempo.
  - Solo en el peor caso ambos algoritmos tardarán lo mismo.
- Es más eficiente que BFS en memoria.
  - Solo requiere mantener en memoria la rama que se está explorando.

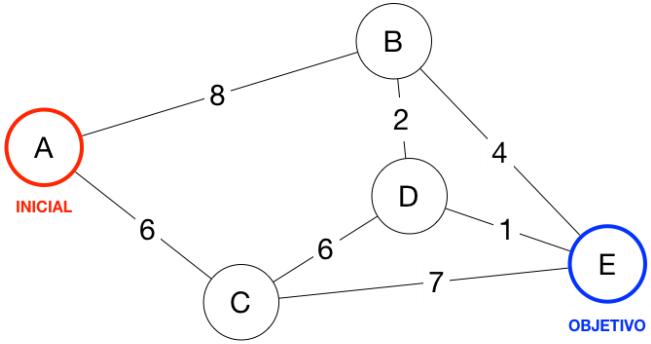


[S.J. Russell, P. Norvig. Artificial Intelligence: A modern approach. Pearson, 2020]

```
DFS(Nodo inicial, Nodo objetivo)
        estado[inicial] = VISITADO
        padre[inicial] = null
        DFS_search(inicial, objetivo)

DFS_search(Nodo u, Nodo objetivo)
        if u==objetivo return TRUE
        for each v in sucesores(u)
                if estado[v] == NOVISITADO
                        estado[v] = VISITADO
                        padre[v] = u
                        return DFS_search(v, objetivo)
        return FALSE
```

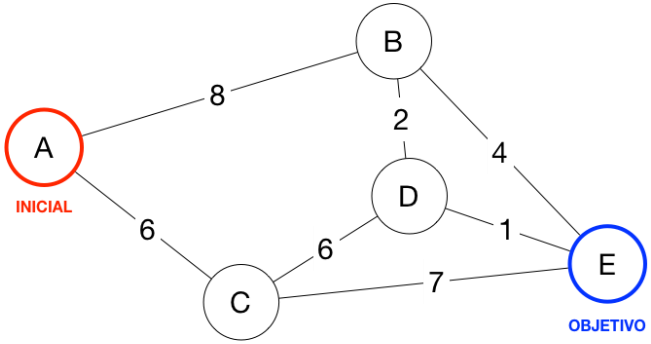
u es un nodo expandido!



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido	Sucesores no visitados	Rama actual	Otras ramas pendientes de explorar (si la actual falla)
0	-	-	[A]	-

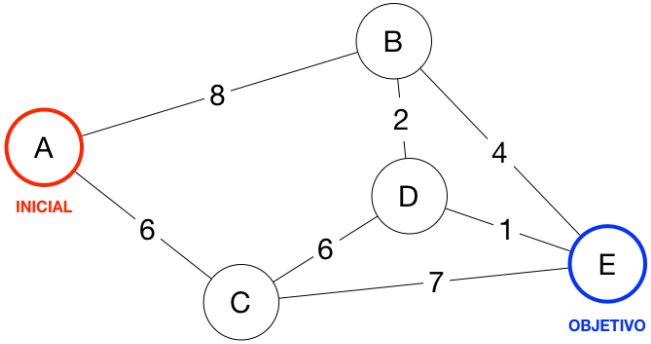
Orden de expansión de sucesores: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido	Sucesores no visitados	Rama actual	Otras ramas pendientes de explorar (si la actual falla)
0	-	-	[A]	-
1	A	B,C	[A,B]	[A,C]

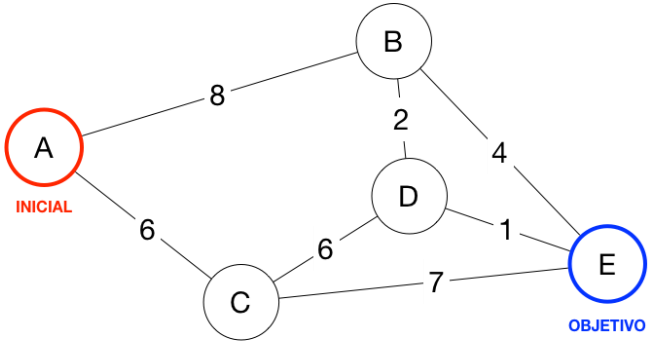
Orden de expansión de sucesores: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido	Sucesores no visitados	Rama actual	Otras ramas pendientes de explorar (si la actual falla)
0	-	-	[A]	-
1	A	B,C	[A,B]	[A,C]
2	B	D,E	[A,B,D]	[A,B,E], [A,C]

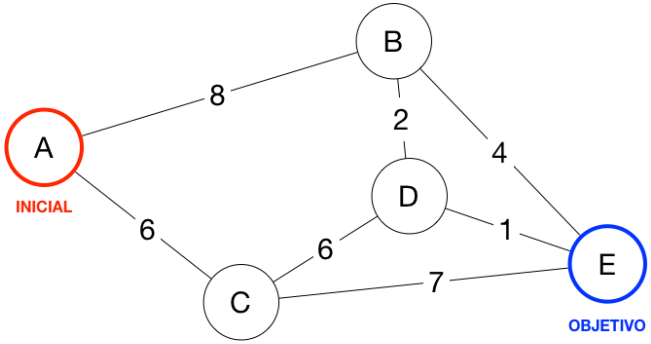
Orden de expansión de sucesores: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido	Sucesores no visitados	Rama actual	Otras ramas pendientes de explorar (si la actual falla)
0	-	-	[A]	-
1	A	B,C	[A,B]	[A,C]
2	B	D,E	[A,B,D]	[A,B,E], [A,C]
3	D	C,E	[A,B,D,C]	[A,B,D,E], [A,B,E], [A,C]

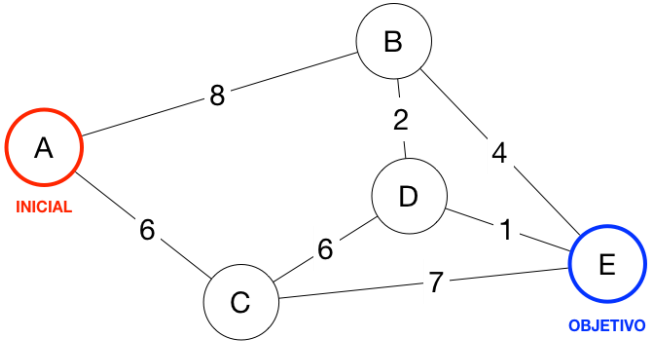
Orden de expansión de sucesores: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo expandido	Sucesores no visitados	Rama actual	Otras ramas pendientes de explorar (si la actual falla)
0	-	-	[A]	-
1	A	B,C	[A,B]	[A,C]
2	B	D,E	[A,B,D]	[A,B,E], [A,C]
3	D	C,E	[A,B,D,C]	[A,B,D,E], [A,B,E], [A,C]
4	C	E	[A,B,D,C,E]	[A,B,D,E], [A,B,E], [A,C]

Orden de expansión de sucesores: orden alfabético (A,B,C, …)



	A	B	C	D	E
h(n)	10	4	5	1	0

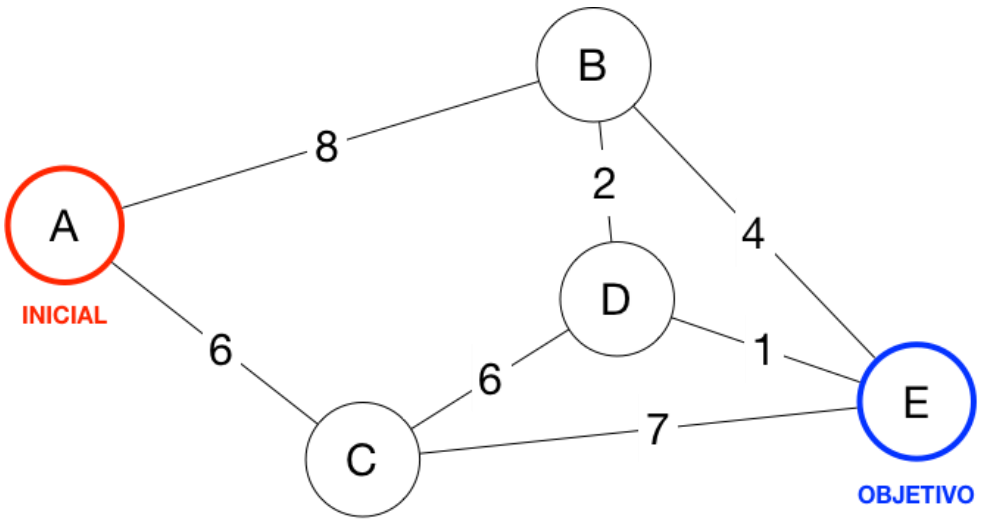
It.	Nodo expandido	Sucesores no visitados	Rama actual	Otras ramas pendientes de explorar (si la actual falla)
0	-	-	[A]	-
1	A	B,C	[A,B]	[A,C]
2	B	D,E	[A,B,D]	[A,B,E], [A,C]
3	D	C,E	[A,B,D,C]	[A,B,D,E], [A,B,E], [A,C]
4	C	E	[A,B,D,C,E]	[A,B,D,E], [A,B,E], [A,C]
5	E	Fin. Camino encontrado: E – C – D – B – A		-

Orden de expansión de sucesores: orden alfabético (A,B,C, …)

No es óptimo!!!
Incluso podría no serlo si los costes fuesen uniformes

Algoritmo de Dijkstra

- Al contrario que BFS y DFS, considera coste heterogéneo (no uniforme) en las aristas
- **Garantiza** encontrar el camino óptimo



	A	B	C	D	E
h(n)	10	4	5	1	0

[Dijkstra. Numerische Mathematik 1: 269-271, 1959]

Russell y Norvig (sección 3.4.2 en la 4ª edición). *"**When actions have different costs**, an obvious choice is to use best-first search where the evaluation function is the cost of the path from the root to the current node. This is called Dijkstra's algorithm by the theoretical computer science community, and uniform-cost search by the AI community. The idea is that while **breadth-first search spreads out in waves of uniform depth** —first depth 1, then depth 2, and so on— **uniform-cost search spreads out in waves of uniform path-cost**. "*

Búsqueda heurística

- **A*** es un algoritmo de **búsqueda heurística offline** capaz de encontrar el camino óptimo.

- Se categorizan como búsqueda offline dado que **TODA** la búsqueda completa se realiza **ANTES** de la ejecución.

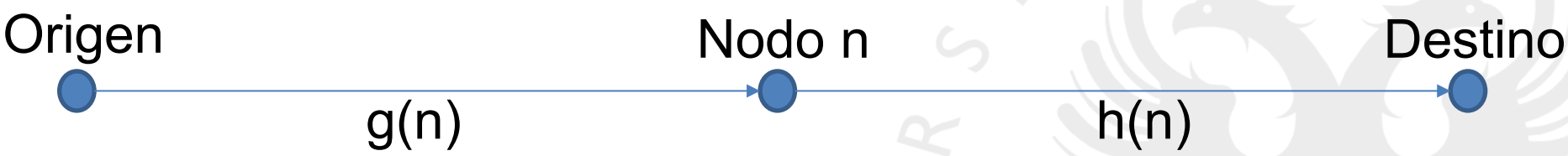
- Otro ejemplo de búsqueda **heurística** offline es el algoritmo **IDA*,** que es una extensión de DFS:
  - Usa profundidad iterativa (lo cual asegura el camino óptimo) + información heurística (para el orden de expansión de los vecinos de un nodo).
  - Por tanto, es más lento que A* pero consume menos memoria.

A*: [Hart, Nilsson, Raphael. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics. 4 (2): 100–107]

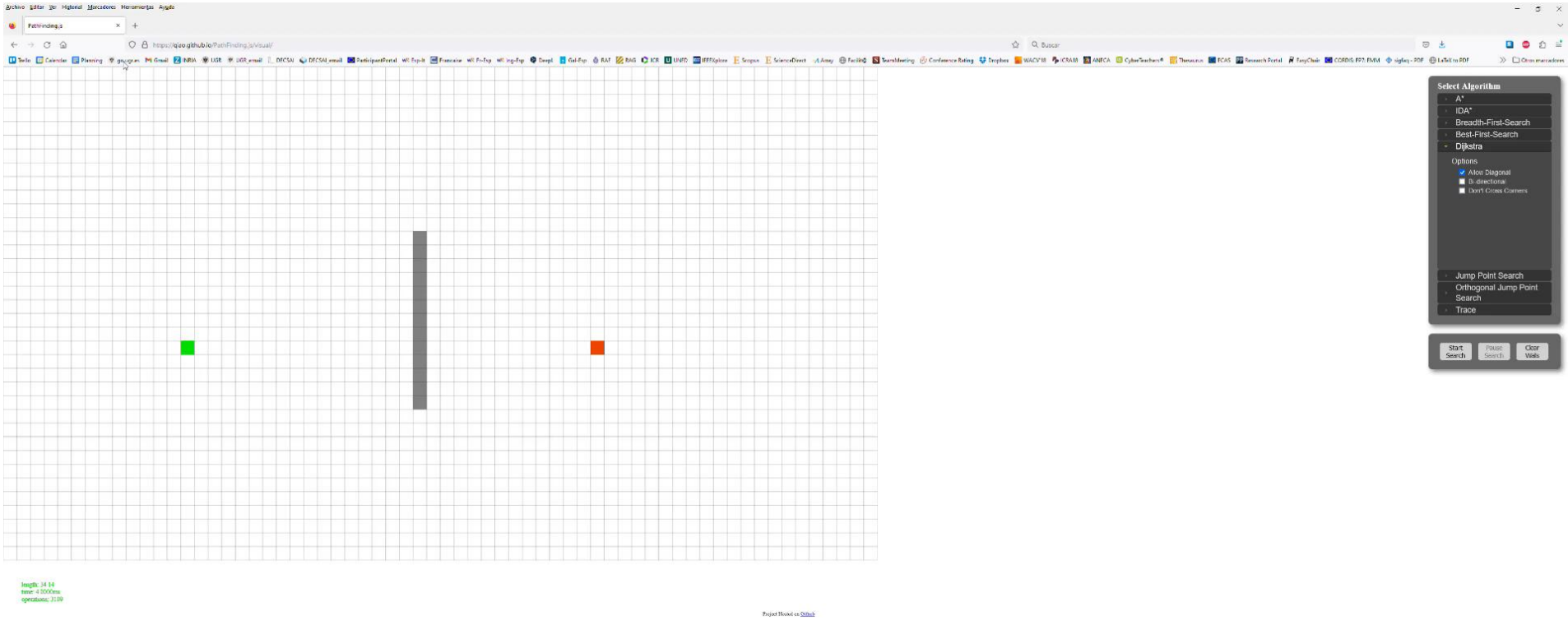
IDA*: [Korf. 1985. Depth-first Iterative-Deepening: An Optimal Admissible Tree Search. Artificial Intelligence. 27: 97–109]

A* = Dijkstra's Algorithm + Greedy Best-First Search

f(n) = g(n) + h(n)



A* vs Dijkstra

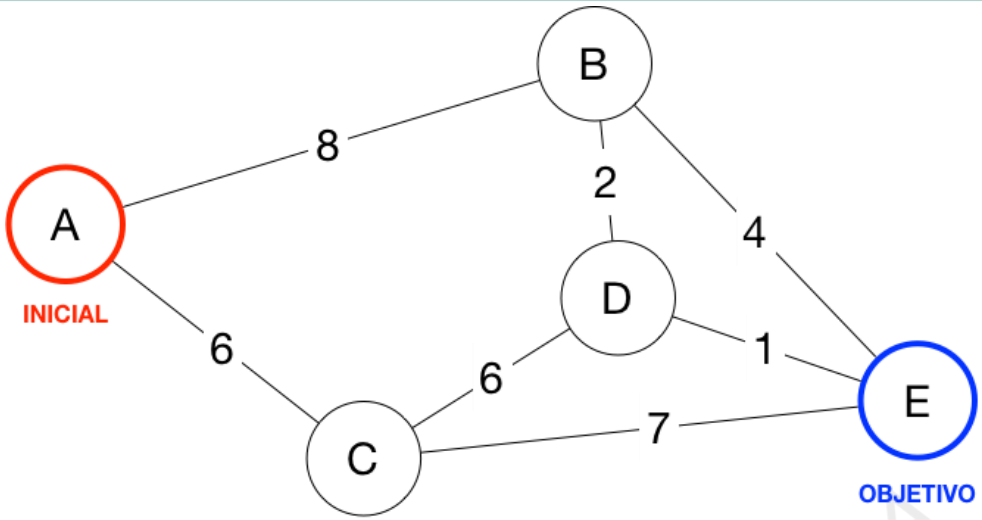


https://qiao.github.io/PathFinding.js/visual/

- *abiertos* = [*inicial*] **# nodos pendientes de visitar**
- cerrados = [] **# nodos ya visitados**
- **while True :**
  - *actual* = ***mejorCandidato***(*abiertos*) **# mejor f(n)**
  - **if** *actual* == *objetivo*:
    - **break # se expande el nodo objetivo, fin.**
  - *abiertos*.remove(*actual*) **and** *cerrados*.add(*actual*)
  - **foreach** *sucesor* **in** ***expandir***(*actual*) : **# vecinos del *nodo***
    - **if** *cerrados*.contains(*sucesor*)
      and ***mejorCaminoA***(*sucesor*): **# menor g(n)**
      - *cerrados*.remove(*sucesor*)
      - *abiertos*.add(*sucesor*) ***# actualizar g(n)***
    - **elif not** *cerrados*.contains(*sucesor*) **and**
      not *abiertos*.contains(*sucesor*) :
      - *abiertos*.add(*sucesor*) ***# nodo no visitado, añadir***
    - **elif** *abiertos*.contains(*sucesor*) **and** **mejorCaminoA**(*sucesor*):
      - *abiertos*.update(*sucesor*) ***# actualizar g(n)***

(nodo) actual es un nodo expandido!

Podría traducirse por “mientras queden nodos por visitar en la lista de abiertos”

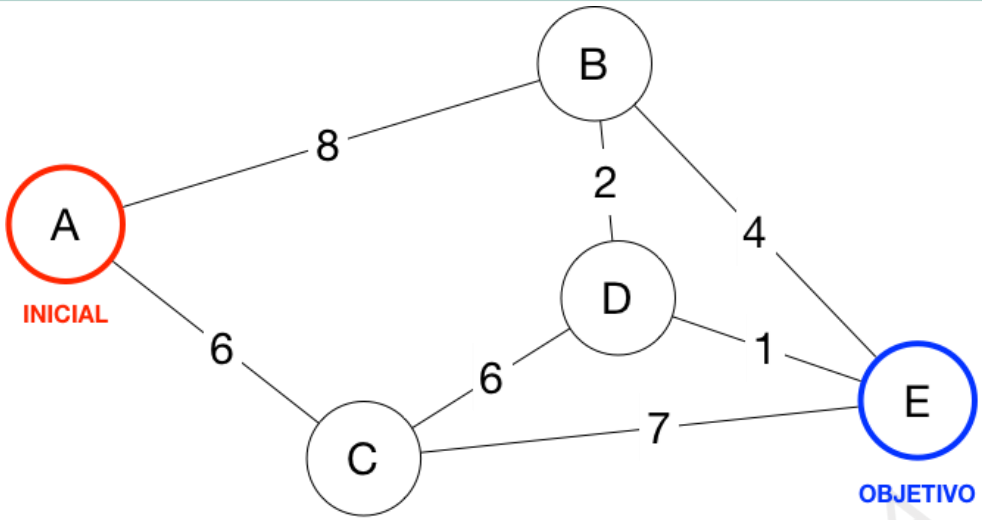


	A	B	C	D	E
h(n)	10	4	5	1	0

¿Es admisible esta heurística?

Si lo es, ¿qué es lo que nos asegura eso?

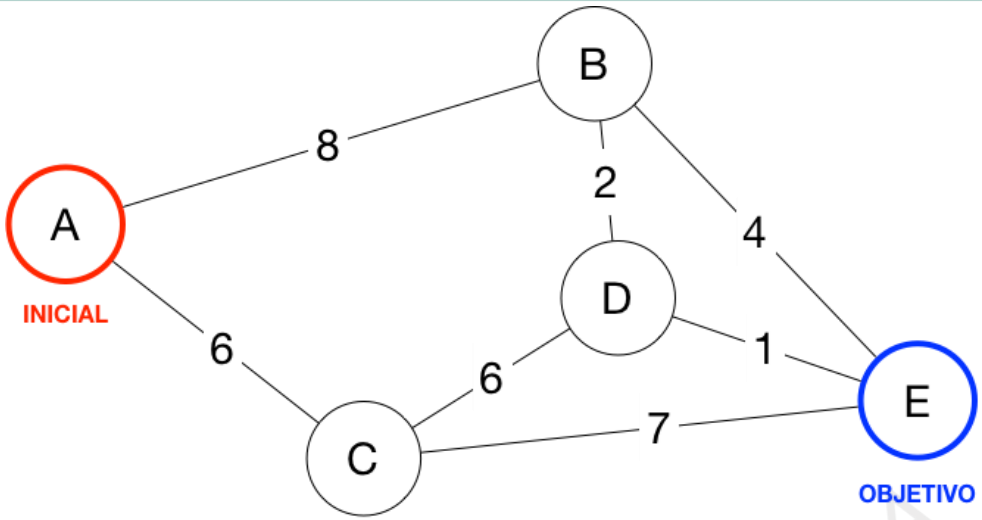
EJEMPLO DE EJECUCIÓN



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Expandir	Sucesores	Abiertos	Cerrados
0			A(0+10, ϕ)	-

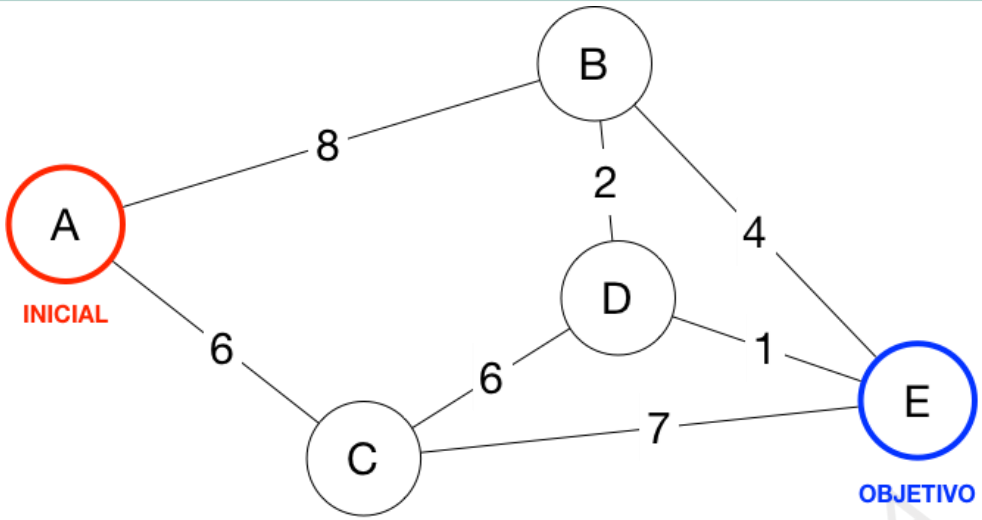
EJEMPLO DE EJECUCIÓN



	A	B	C	D	E
h(n)	10	4	5	1	0

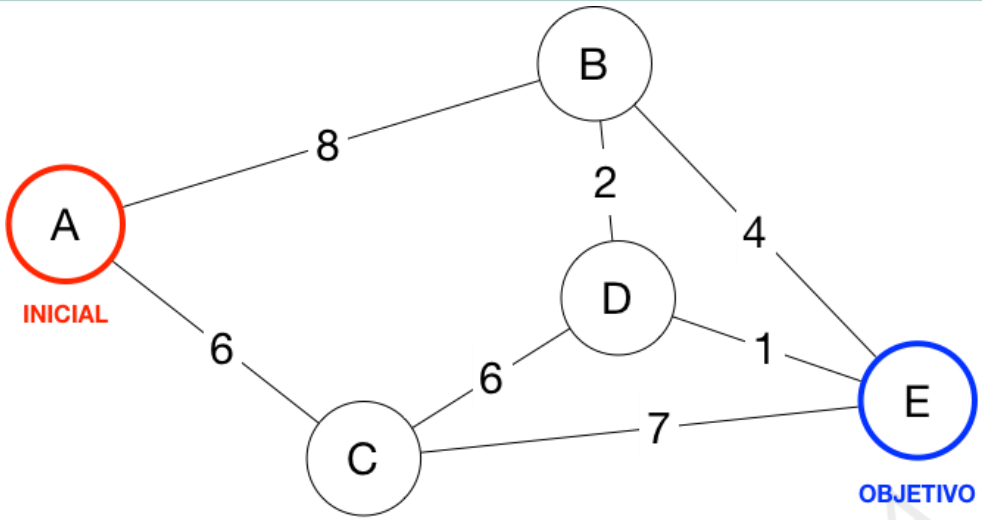
It.	Expandir	Sucesores	Abiertos	Cerrados
0			A(0+10, ϕ)	-
1	A (-)	B(8+4, A), C(6+5, A)	B(8+4, A), C(6+5, A)	A

EJEMPLO DE EJECUCIÓN



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Expandir	Sucesores	Abiertos	Cerrados
0			A(0+10, ϕ)	-
1	A (-)	B(8+4, A), C(6+5, A)	B(8+4, A), C(6+5, A)	A
2	C (A)	D(12+1,C), E(13+0,C)	B(8+4,A), D(12+1,C), E(13+0,C)	A,C



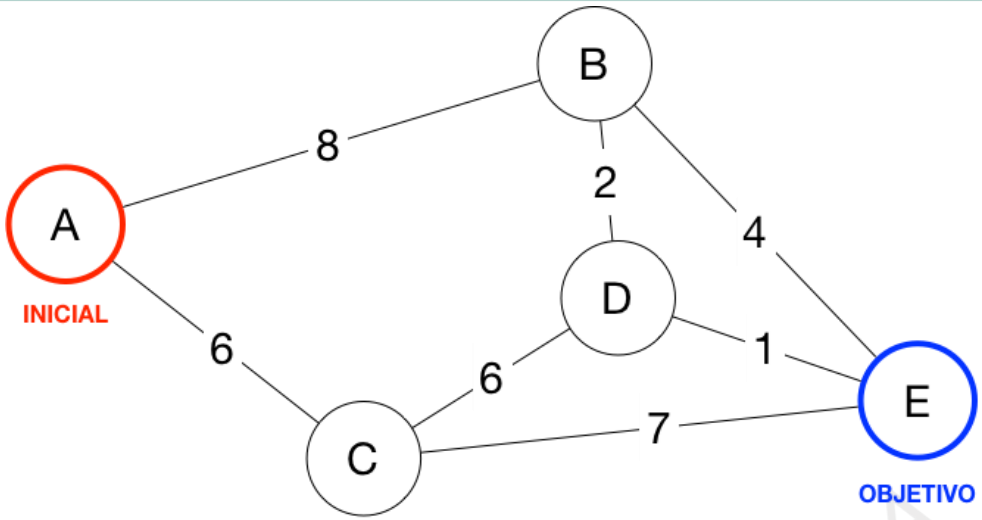
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Expandir	Sucesores	Abiertos	Cerrados
0			A(0+10, ϕ)	-
1	A (-)	B(8+4, A), C(6+5, A)	B(8+4, A), C(6+5, A)	A
2	C (A)	D(12+1,C), E(13+0,C)	B(8+4,A), D(12+1,C), E(13+0,C)	A,C
3	B (A)	D(10+1,B), E(12+0,B)	~~D(12+1,C), E(13+0,C)~~, D(10+1,B), E(12+0,B)	A,C,B

elif *abiertos*.contains(*sucesor*) **and** **mejorCaminoA**(*sucesor*):

- *abiertos*.update(*sucesor*) ***# actualizar g(n)***

EJEMPLO DE EJECUCIÓN



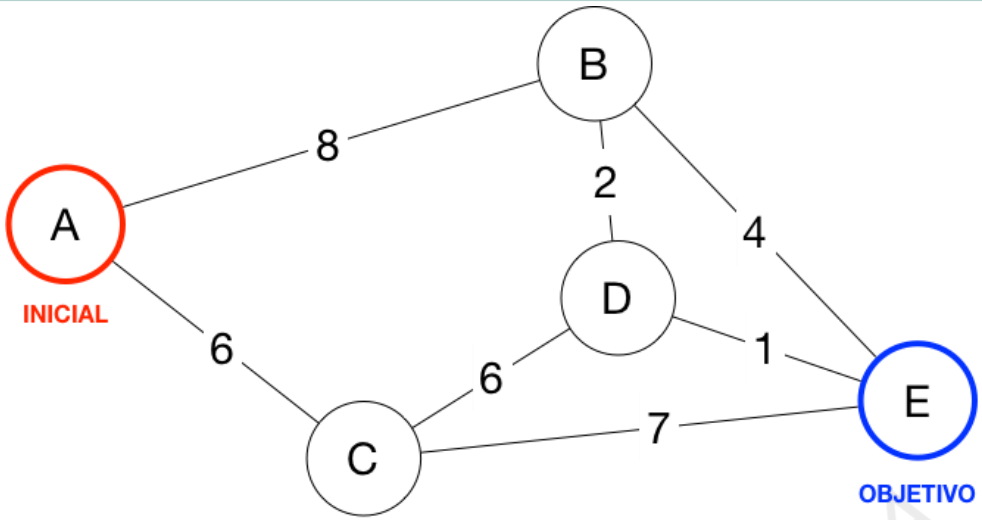
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Expandir	Sucesores	Abiertos	Cerrados
0			A(0+10, ϕ)	-
1	A (-)	B(8+4, A), C(6+5, A)	B(8+4, A), C(6+5, A)	A
2	C (A)	D(12+1,C), E(13+0,C)	B(8+4,A), D(12+1,C), E(13+0,C)	A,C
3	B (A)	D(10+1,B), E(12+0,B)	~~D(12+1,C), E(13+0,C)~~, D(10+1,B), E(12+0,B)	A,C,B
4	D (B)	~~C(16+5,D)~~, E(11+0,D)	~~E(12+0,B)~~, E(11+0,D)	A,C,B,D

cerrados.contains(*sucesor*)
PERO NO
mejorCaminoA(*sucesor*)

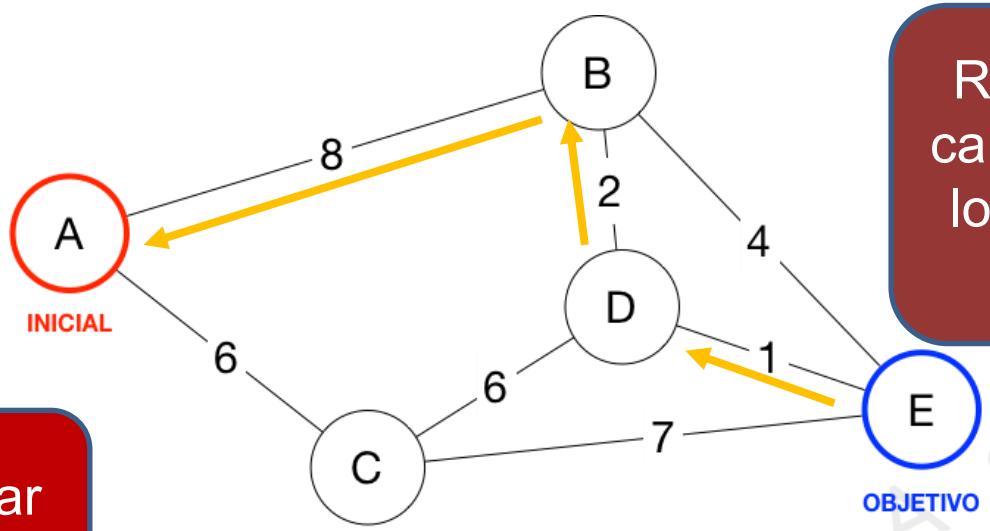
elif *abiertos*.contains(*sucesor*) **and** **mejorCaminoA**(*sucesor*):
- *abiertos*.update(*sucesor*) ***# actualizar g(n)***

EJEMPLO DE EJECUCIÓN



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Expandir	Sucesores	Abiertos	Cerrados
0			A(0+10, ϕ)	-
1	A (-)	B(8+4, A), C(6+5, A)	B(8+4, A), C(6+5, A)	A
2	C (A)	D(12+1,C), E(13+0,C)	B(8+4,A), D(12+1,C), E(13+0,C)	A,C
3	B (A)	D(10+1,B), E(12+0,B)	~~D(12+1,C), E(13+0,C)~~, D(10+1,B), E(12+0,B)	A,C,B
4	D (B)	~~C(16+5,D)~~, E(11+0,D)	~~E(12+0,B)~~, E(11+0,D)	A,C,B,D
5	E (D)	Fin		A,C,B,D,E



Reconstruimos el camino recorriendo los punteros a los nodos padre

¡Hay que guardar el nodo padre!

	A	B	C	D	E
h(n)	10	4	5	1	0

Nodo	Nodo padre	Coste del arco	Ruta	Coste de la ruta
E (Objetivo)	D	1	E - D	1
D	B	2	E – D - B	1+2 = 3
B	A	8	**E – D – B – A**	1+2+8 = **11**
A (Inicial)				

Para más información:

Introduction to the A* algorithm:
https://www.redblobgames.com/pathfinding/a-star/introduction.html

Aplicaciones visuales para evaluar distintos algoritmos en grids:
https://qiao.github.io/PathFinding.js/visual/
https://pathfinding-visualizer-nu.vercel.app/

1) Verificad manualmente, con distintos mapas sencillos, que vuestras soluciones son correctas. Para ello, inecesitáis comprender los algoritmos!
   - Recordad que, casi con total seguridad, los mapas de evaluación serán diferentes a los que os proporcionamos con la práctica.
   - ¿Qué podríamos intentar verificar?
     - Orden de expansión de nodos y criterios de desempate.
     - Que las rutas calculadas por A* son correctas.
     - No se fuerza a coger todas las monedas en el mapa.
     - El código funciona con más de una llave en el mapa.
     - Si disponemos del máximo de monedas en el inventario, no se recogen más.
     - Etc.
   - ➢ Elementos clave:
     - Representación completa del estado.
     - Estructuras de datos adecuadas.
     - Verificación/depuración del código con mapas representando casuísticas diversas.

2) Si vuestros resultados no son correctos, podéis simplificar el problema para depurar el código.
   - P.ej., eliminad catapultas y verificad que vuestro algoritmo de búsqueda, al menos, os permite llegar al portal en el número correcto de pasos.